MINI-TRANSFORMER + RAG

Guide d’apprentissage : préparation du texte avant le mécanisme d’attention

Objectif

Ce document récapitule le pipeline construit avant l’implémentation de l’attention : chargement du corpus, tokenisation, construction du vocabulaire, attribution des IDs, encodage des phrases, création de tenseurs et conversion des IDs en embeddings PyTorch.

Pipeline général

Corpus brut	Tokenisation	Vocabulaire	IDs	Embeddings
Phrases	Tokens	Token ↔ ID	Entiers	Vecteurs d_model

Version pédagogique basée sur une tokenisation simple par mots. L’objectif est de comprendre chaque transformation avant d’ajouter l’attention, les masques et l’information positionnelle.

1. Vue d’ensemble

Un Transformer ne réalise pas ses calculs directement sur les mots écrits. Il manipule des tenseurs numériques. Nous devons donc convertir le texte brut en tokens, les tokens en identifiants entiers, puis les identifiants en vecteurs.

Dans notre projet, nous commençons par un tokenizer simple en français. Ce choix est volontaire : il permet d’apprendre les mécanismes essentiels avant d’explorer des tokenizers plus avancés comme BPE.

Nous ne supprimons pas les stop words. Des mots comme « ne », « pas », « dans » ou « que » peuvent changer le sens d’une phrase. Pour un modèle de langage, on conserve normalement ces mots et le modèle apprend leurs relations pendant l’entraînement.

2. Charger le corpus

Le corpus est l’ensemble des phrases utilisées pour construire le vocabulaire et, plus tard, entraîner le modèle. Dans notre projet, le fichier est situé à `src/small_agentic_rag/data/corpus.txt`.

```
from pathlib import Path

corpus_path = Path("src/small_agentic_rag/data/corpus.txt")

with corpus_path.open("r", encoding="utf-8") as file:
    corpus = [
        line.strip()
        for line in file
        if line.strip()
    ]

print("Nombre de phrases :", len(corpus))
print("Première phrase :", corpus[0])
```

Pourquoi ces opérations ?

• Path permet de manipuler le chemin du fichier de manière lisible.

• encoding='utf-8' assure une lecture correcte des accents.

• strip() retire les espaces autour de la phrase.

• La condition if line.strip() ignore les lignes vides.

À ce stade, `corpus` est une liste de chaînes de caractères. Aucun ID n’est encore associé aux mots.

3. Tokenisation

La tokenisation découpe une phrase en une séquence de tokens. Dans notre première version, un token correspond approximativement à un mot séparé par des espaces.

```
text = "Le chat ne mange pas."
tokens = text.lower().split()

print(tokens)
# ['le', 'chat', 'ne', 'mange', 'pas.']
```

`lower()` met les caractères en minuscules. `split()` découpe sur les espaces et gère mieux les espaces multiples que `split(' ')`.

Limite actuelle : la ponctuation

Le point reste attaché au token « pas. ». Il sera donc différent de « pas ». C’est une limite connue de notre tokenizer minimal. Nous pourrons ensuite séparer la ponctuation ou étudier un tokenizer par sous-mots comme BPE.

4. Construire le vocabulaire

Nous rassemblons d’abord tous les tokens, répétitions comprises, puis nous construisons la collection des tokens uniques.

```
all_tokens = []

for line in corpus:
    all_tokens.extend(line.lower().split())
```

```
unique_tokens = sorted(set(all_tokens))
```

`set(all_tokens)` supprime les doublons. `sorted(...)` impose un ordre reproductible pour attribuer les IDs. Ce tri ne conserve pas l'ordre des mots dans les phrases : cet ordre est conservé dans chaque séquence encodée.

Le vocabulaire est un dictionnaire qui associe chaque token à un entier. Par exemple, « chat » peut recevoir l'ID 1 et « le » l'ID 3. Les valeurs précises n'ont pas de sens linguistique en elles-mêmes.

Token spécial

```
special_tokens = ["<unk>"]

vocab = {
    token: idx
    for idx, token in enumerate(special_tokens)
}

for token in sorted(set(all_tokens)):
    if token not in vocab:
        vocab[token] = len(vocab)
```

<unk> signifie « unknown ». Il représente un token absent du vocabulaire. Son ID est ici 0. Cette convention permet à `encode()` de traiter un mot inconnu sans provoquer d'erreur.

Le vocabulaire doit rester stable pendant l'entraînement et l'utilisation du modèle. Si les IDs changent après l'apprentissage, les lignes de la table d'embeddings ne correspondront plus aux mêmes tokens.

5. À quoi sert Counter ?

Counter compte les occurrences des tokens. Il doit être appliqué à la liste qui contient encore les répétitions.

```
from collections import Counter

counter = Counter(all_tokens)

print("Occurrences de chat :", counter["chat"])
print("Tokens les plus fréquents :", counter.most_common(10))
```

Si Counter est appliqué après avoir supprimé les doublons, chaque token aura une fréquence de 1. Counter n'est pas obligatoire pour créer le vocabulaire, mais il est utile pour analyser le corpus et comprendre la distribution des mots.

6. Encoder et décoder

L'encodage convertit le texte en IDs grâce au vocabulaire. Le décodage fait l'opération inverse à l'aide d'un dictionnaire ID → token.

```
id_to_token = {
    idx: token
    for token, idx in vocab.items()
}

def encode(text):
    tokens = text.lower().split()
    unk_id = vocab["<unk>"]
    return [vocab.get(token, unk_id) for token in tokens]

def decode(ids):
    return " ".join(
        id_to_token.get(int(idx), "<unk>")
        for idx in ids
    )

phrase = "le chat mange"
ids = encode(phrase)

print("IDs :", ids)
print("Texte reconstruit :", decode(ids))
```

Si « le » vaut 3, « chat » vaut 1 et « mange » vaut 4, alors « le chat mange » devient [3, 1, 4]. Les IDs sont des symboles numériques : l'écart entre les nombres n'exprime ni distance sémantique ni ordre de mots.

Le texte reconstruit reflète les règles du tokenizer. Comme notre version passe en minuscules et conserve la ponctuation attachée, le décodage peut ne pas reproduire exactement la casse ou le format original.

7. Convertir les IDs en tenseur

PyTorch utilise des tenseurs pour effectuer les calculs. Une liste d'IDs doit être convertie en tenseur d'entiers, généralement de type `torch.long` pour une couche `nn.Embedding`.

```
import torch

ids = encode("le chat mange")
token_ids = torch.tensor(ids, dtype=torch.long)

print(token_ids)
print(token_ids.shape)  # torch.Size([3])
```

La forme (3,) signifie que la séquence contient trois tokens. Pour traiter plusieurs phrases ensemble, nous ajouterons une dimension de batch : (batch_size, sequence_length). Les phrases de longueurs différentes nécessitent souvent du padding et un masque.

8. Des IDs aux embeddings

Une couche d'embedding est une table de paramètres entraînables. Chaque ID sélectionne une ligne de cette table. Si le vocabulaire contient V tokens et que la dimension choisie est d_model, la table a la forme (V, d_model).

```
import torch
import torch.nn as nn

vocab_size = len(vocab)
d_model = 64

embedding = nn.Embedding(
    num_embeddings=vocab_size,
    embedding_dim=d_model
)

token_ids = torch.tensor(
    encode("le chat mange"),
    dtype=torch.long
)

vectors = embedding(token_ids)

print("Forme des IDs :", token_ids.shape)  # (3,)
print("Forme des embeddings :", vectors.shape)  # (3, 64)
```

La phrase contient trois tokens et chaque token est représenté par un vecteur de 64 nombres. La sortie a donc la forme (sequence_length, d_model), ici (3, 64). Avec un batch, la forme habituelle devient (batch_size, sequence_length, d_model).

Les vecteurs initiaux sont généralement initialisés aléatoirement. Ils ne connaissent pas automatiquement le sens des mots : leurs valeurs seront ajustées au cours de l'entraînement en fonction de la tâche et du gradient.

Ne pas confondre token, ID et embedding

Élément	Exemple	Rôle
Token	chat	Unité de texte
ID	1	Identifiant discret du token
Embedding	[-0.12, 0.31, ...]	Vecteur numérique appris

9. Que reste-t-il avant l'attention ?

• Batching et padding : les phrases ont des longueurs variables. Pour les regrouper, on peut ajouter un token <pad> et construire un masque de padding.

• Information de position : les embeddings de tokens seuls ne disent pas où se trouve chaque token dans la séquence. Nous étudierons notamment RoPE, qui agit sur les représentations de requêtes et de clés.

• Masque causal : pour un modèle autorégressif qui prédit le prochain token, une position ne doit pas accéder aux tokens futurs.

• Objectif d'entraînement : il faudra définir les entrées et les cibles décalées d'un token pour apprendre la prédiction du token suivant.

L'attention prend en entrée des vecteurs et calcule comment chaque position peut combiner l'information provenant d'autres positions. Elle ne travaille pas directement sur les mots écrits ni sur les IDs.

10. Organisation suggérée

```
src/small_agentic_rag/
■■■ data/
■   ■■■ corpus.txt
■■■ tokenizer.py       # vocabulaire, encode(), decode()
■■■ embeddings.py      # nn.Embedding
■■■ attention.py       # prochaine étape
■■■ transformer.py     # assemblage progressif

tests/
■■■ test_tokenizer.py
■■■ test_embeddings.py
```

Adapte les chemins à la structure réelle de ton dépôt. La séparation des responsabilités est le point important : le tokenizer produit des IDs, l'embedding convertit ces IDs en vecteurs et l'attention exploite ensuite ces vecteurs.

11. Checklist de validation

• ☐ Le corpus est chargé correctement et les lignes vides sont ignorées.

• ☐ Les stop words sont conservés.

• ☐ La tokenisation est cohérente.

• ☐ Chaque token possède un ID unique et stable.

• ☐ Le token gère les tokens inconnus.

• ☐ encode() convertit une phrase en IDs.

• ☐ decode() permet de vérifier le mapping inverse.

• ☐ Les IDs sont convertis en tenseur torch.long.

• ☐ nn.Embedding renvoie la forme attendue.

• ☐ Le vocabulaire est sauvegardé avec le modèle.

Conclusion : nous avons construit la chaîne qui va du texte brut à une séquence de vecteurs numériques. C'est la représentation que nous allons utiliser pour commencer l'implémentation de l'attention. La prochaine étape sera d'étudier les projections Q, K et V, le calcul des scores d'attention, le softmax et le masque causal.